libDwm

binary serialization and deserialization

Version 1.0.0

Last Revision: October 22, 2025 12:51am -04:00

Contents

Chapter 1

Introduction

1.1 Motivation

Efficiency I did not want conversions between textual and native representations for arithmetic types. For example, if I need to stream an unsigned 32-bit integer of value 0xFFFFFFFF, I should only need to transfer 4 bytes, not 10 or more.

Portability Since network transport was a primary goal, I was motivated to provide schemes that work across fairly common hardware platforms and UNIX operating systems.

Flexibility I/O facilities should be flexible enough to accomodate instances of common class templates from the standard library, particularly containers like `vector`, `map`, et. al. They should also be flexible enough to accomodate user-defined types without a tremendous amount of work.

1.2 History

libDwm was started around 1998. Needless to say, C++ was a very different language in 1998 versus 2025. And the standard library has grown tremendously since 1998.

The I/O facilities in libDwm started appearing in the early 2000's (around the time we got C++03). Since that time, it has evolved a bit with each new version of the C++ standard, and is about to see a sea change with C++26 reflection (P2996 and P3394 in particular).

1.3 Primary I/O classes

1.4 Directly supported types

1.5 Supported `std` class templates

1.6 Concepts

Chapter 2

Design

2.1 A primary I/O class for each source/sink type

The library divides I/O functionality into a primary I/O class for a given category of sources/sinks.

Primary I/O class	Source type	Sink type	Description
Dwm::StreamIO	std::istream &	std::ostream &	references to C++ iostreams
Dwm::FileIO	FILE *	FILE *	FILE created with fopen() and friends
Dwm::DescriptorIO	int	int	UNIX descriptors
Dwm::BZ2IO	BZFILE *	BZFILE *	BZFILE created with BZ2_bzopen()
Dwm::GZIO	gzFile	gzFile	gzFile created with gzopen()

Each of the primary I/O classes provides a set of static functions and function templates to read and write data.

2.2 Conceptual interfaces

Conceptually, the I/O classes provide four primary interfaces to read and write from/to their associated source/sink type, as depicted in table 2.1. There are single-object interfaces (ReadFunc and WriteFunc) and multi-object variadic interfaces (VariadicReadFunc and VariadicWriteFunc). The multi-object variadic interfaces just call to the single-object interfaces for each object.

Table 2.1: Conceptual read/write functions in all primary I/O classes

Read/write functions (conceptual)

```
template <typename T>
static ReadReturnType & ReadFunc(SourceType source, T & t);
template <typename T>
static WriteReturnType WriteFunc(SinkType sink, const T & t);
template <typename...  Args>
static ReadReturnType VariadicReadFunc(SourceType source, Args & ...args);
template <typename...  Args>
static WriteReturnType VariadicWriteFunc(SinkType sink, const Args & ...args);
```

Table 2.2 maps the generic conceptual interface to each of the primary I/O classes.

Table 2.2: Conceptual interface mapping to the primary I/O classes

	Dwm I/O class				
	StreamIO	FileIO	DescriptorIO	BZ2IO	GZIO
ReadFunc	Read	Read	Read	BZRead	Read
VariadicReadFunc	ReadV	ReadV	ReadV	BZReadV	ReadV
ReadReturnType	std::istream &	size_t	ssize_t	int	int
SourceType	std::istream &	FILE *	int	BZFILE *	gzFile
WriteFunc	Write	Write	Write	BZWrite	Write
VariadicWriteFunc	WriteV	WriteV	WriteV	BZWriteV	WriteV
WriteReturnType	std::ostream &	size_t	ssize_t	int	int
SinkType	std::ostream &	FILE *	int	BZFILE *	gzFile

2.3 Categories of interface overloads in actual implementation

2.3.1 Regular functions for directly supported types

`read` and `write` of directly supported types is done with regular function overloads.

2.3.2 Function templates for STL container and other standard class templates

Overloaded function templates are used to read and write types instantiated via standard class templates, where the parameter types of the standard class templates are either directly supported types, types that meet the conceptual requirements of a readable and writable class, or instances of supported standard templates using any of the aforementioned types in type parameters. In other words, supported standard containers of any depth can be read and written, as long as the leaf types are directly supported types or types that meet the conceptual requirements of a readable and writable class for the I/O class of interest.

This is probably best explained via some example readable and writable types:

```
std::vector<std::string>
std::map<uint32_t,std::vector<std::string>>
std::map<uint32_t,std::vector<std::map<uint8_t,std::string>>>
```

2.3.3 Function templates for types that meet the conceptual requirements of a given primary I/O class

Overloaded function templates with concept requirements are used to read and write types that meet the concept requirements.

Dwm I/O class	**Read concept**	**Write concept**
StreamIO	Dwm::HasStreamRead<T>	Dwm::HasStreamWrite<T>
FileIO	Dwm::HasFileRead<T>	Dwm::HasFileWrite</t>
DescriptorIO	Dwm::HasDescriptorRead<T>	Dwm::HasDescriptorWrite<T>
BZ2IO	Dwm::HasBZRead<T>	Dwm::HasBZWrite<T>
GZIO	Dwm::HasGZRead<T>	Dwm::HasGZWrite<T>

2.4 Dwm::StreamIO read/write functions

2.4.1 Dwm::StreamIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::StreamIO	`template <typename T>` `static std::istream & Read(std::istream & is, T & t);`
	`template <typename T>` `static std::ostream & Write(std::ostream & os, const T & t);`
	`template <typename... Args>` `static std::istream & ReadV(std::istream & is, Args & ...args);`
	`template <typename... Args>` `static std::ostream & WriteV(std::ostream & os, const Args & ...args);`

Conceptually...

`template <typename T>`
`static std::istream & Read(std::istream & is, T & t)`

- Reads the contents of `t` from `is` and returns `is`.

`template <typename T>`
`static std::Write(std::ostream & os, const T & t)`

- Writes the contents of `t` to `os` and returns `os`.

`template <typename... Args>`
`static std::istream & ReadV(std::istream & is, Args & ...args)`

- Reads the contents of all `args` (in order) from `is` and returns `is`.

`template <typename... Args>`
`static std::ostream & WriteV(std::ostream & os, const Args & ...args)`

- Writes the contents of all `args` (in order) to `os` and returns `os`.

2.5 Dwm::FileIO read/write functions

2.5.1 Dwm::FileIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::FileIO	`template <typename T>` `static size_t Read(FILE *f, T & t);`
	`template <typename T>` `static size_t Write(FILE *f, const T & t);`
	`template <typename... Args>` `static size_t ReadV(FILE *f, Args & ...args);`
	`template <typename... Args>` `static size_t WriteV(FILE *f, const Args & ...args);`

2.6 Dwm::DescriptorIO read/write functions

2.6.1 Dwm::DescriptorIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::DescriptorIO	template <typename T> static ssize_t Read(int fd, T & t);
	template <typename T> static ssize_t Write(int fd, const T & t);
	template <typename... Args> static ssize_t ReadV(int fd, Args & ...args);
	template <typename... Args> static ssize_t WriteV(int fd, const Args & ...args);

2.7 Dwm::BZ2IO read/write functions

2.7.1 Dwm::BZ2IO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::BZ2IO	template <typename T> static int BZRead(BZFILE *bzf, T & t);
	template <typename T> static int BZWrite(BZFILE *bzf, const T & t)
	template <typename... Args> static int BZReadV(BZFILE *bzf, Args & ...args);
	template <typename T> static int BZWriteV(BZFILE *bzf, const Args & ...args)

2.8 Dwm::GZIO read/write functions

2.8.1 Dwm::GZIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::GZIO	template <typename T> static int Read(gzFile gzf, T & t);
	template <typename T> static int Write(gzFile gzf, const T & t);
	template <typename... Args> static int ReadV(gzFile gzf, Args & ...args);
	template <typename... Args> static int Write(gzFile gzf, const Args & ...args);

2.9 Member function overloads

libDwm uses member function overloading to allow the user to use the same function names to read or write different types from/to a source/sink. This is also critical to supporting instances of class templates.

2.10 Member function templates

libDwm uses function templates and function template overloads to provide the same interfaces (the same member function names) to read or write instances of standard class templates from/to a source/sink (`std::vector`, `std::pair`, `std::tuple`, `std::map`, et. al.).

2.11 Concepts

libDwm uses concepts in many places to aid in overload resolution and to document the requirements of user-defined types.

2.12 Support for user-designed types

2.12.1 Before C++26

In general, code utilized with pre-C++26 compilers and standard libraries requires a pair of member functions to be added to any user-defined type for a given source/sink pair. Much of the time, these functions are fairly simple to implement, particuarly for C++ iostreams (`std::istream` as a source, `std::ostream` as a sink). This is especially true for user-defined types that only contain types already supported by the corresponding primry I/O class (for C++ iostreams, that would be `Dwm::StreamIO`).

2.12.2 C++26 and beyond

With C++26 features (particularly P2996 and P3394), it becomes possible to elegantly handle serialization and deserialization of many user defined types with no changes to user code. It also allows annotations of types and variables that can control the serialization and deserialization of a user defined type (for example, skipping a data member of a class during serialization and deserialization). Work here has already begun, using a slightly modified fork of the bloomberg clang implementation.

Chapter 3

Examples

3.1 Write and read a `string` to/from an `iostream`

The program below writes a `std::string` to a `std::stringstream`, then reads a `std::string` from the `std::stringstream`. The program in listing 3.1 returns `0` if the string we read matches the string we wrote.

```
1  #include <sstream>
2  #include "DwmStreamIO.hh"
3
4  int main(int argc, char *argv[])
5  {
6    using std::string, std::stringstream;
7    bool          success{false};
8    string        writeString("Hello, world!"), readString;
9    stringstream  ss;
10   if (Dwm::StreamIO::Write(ss, writeString)) {
11     if (Dwm::StreamIO::Read(ss, readString)) {
12       success = (readString == writeString);
13     }
14   }
15   return success ? 0 : 1;
16 }
```

Listing 3.1: Write and read a `string` to/from an `iostream`

3.2 Write and read a map<string,string> to/from an iostream

The program in listing 3.2 writes a std::map<std::string,std::string> to a std::stringstream, then reads a std::map<std::string,std::string> from the std::stringstream and returns 0 if the map we read matches the map we wrote. This is an example of our support for standard containers holding supported types (std::string is an explicitly supported type).

```
1  #include <sstream>
2  #include "DwmStreamIO.hh"
3
4  int main(int argc, char *argv[])
5  {
6    using stringmap = std::map<std::string,std::string>;
7    stringmap  m1 {
8      { "apple",    "fruit"     },
9      { "cat",      "feline"    },
10     { "broccoli", "vegetable" },
11     { "dog",      "canine"    },
12     { "kiwi",     "fruit"     },
13     { "cow",      "bovine"    }
14   };
15   bool               success{false};
16   std::stringstream  ss;
17   if (Dwm::StreamIO::Write(ss, m1)) {
18     stringmap        m2;
19     if (Dwm::StreamIO::Read(ss, m2)) {
20       success = (m2 == m1);
21     }
22   }
23   return success ? 0 : 1;
24 }
```

Listing 3.2: Write and read a map<string,string> to/from an iostream

3.3 Turtles all the way down

The program in listing 3.3 writes a `std::map<std::string,std::vector<std::string>>` to a `std::stringstream`, then reads a `std::map<std::string,std::vector<std::string>>` from the `std::stringstream` and returns `0` if the map we read matches the map we wrote. This is an example of our support for standard containers and supported types being "turtles all the way down": our support is recursive to any depth.

```
#include <sstream>
#include "DwmStreamIO.hh"

int main(int argc, char *argv[])
{
  using container = std::map<std::string,std::vector<std::string>>;
  container  m1 {
    { "fruit",     { "apple", "banana", "kiwi", "orange" }             },
    { "canine",    { "husky", "labrador", "poodle", "wolf", "collie" } },
    { "feline",    { "cougar", "lion", "panther" }                     },
    { "vegetable", { "broccoli", "carrot", "squash" }                  }
  };

  bool               success{false};
  std::stringstream  ss;
  if (Dwm::StreamIO::Write(ss, m1)) {
    container  m2;
    if (Dwm::StreamIO::Read(ss, m2)) {
      success = (m2 == m1);
    }
  }
  return success ? 0 : 1;
}
```

Listing 3.3: Write and read a `map<string,vector<string>>` to/from an `iostream`

3.4 Integrating a user-defined type

Listing 3.4 shows a user defined type `MyType` with three simple data members: `id`, `name` and `category`.

```
#include <cstdint>
#include <string>

struct MyType {
  uint32_t     id;
  std::string  name;
  std::string  category;
};
```

Listing 3.4: `MyType` without explicit `Dwm::StreamIO` support

3.4.1 With C++26 reflection

With C++26 reflection, all of the primary I/O classes can read and write instances of `MyType` from/to their associated source/sink with no changes to `MyType`. The primary I/O classes will do what one would expect: read and write each of the members of `MyType`, in the order in which they appear in the declaration of `MyType`.

3.4.2 Without C++26 reflection

Without C++26 reflection, two members need to be added to `MyType` for each primary I/O class you wish to use with `MyType`. For example, listing 3.5 adds support for `Dwm::StreamIO` by adding the `Read(std::istream &)` and `Write(std::ostream &) const` member functions for the case where C++26 reflection is not available (`DWM_CAN_USE_REFLECTION` is not defined). Note that these member functions are simple in this case because our member data types are directly supported by `Dwm::StreamIO` and hence we can use the `ReadV()` and `WriteV()` members of `Dwm::StreamIO` to implement these members.

```
#include "DwmStreamIO.hh"

struct MyType {
  uint32_t     id;
  std::string  name;
  std::string  category;

#if ! defined(DWM_CAN_USE_REFLECTION)
  std::istream & Read(std::istream & is)
  { return Dwm::StreamIO::ReadV(is, id, name, category); }

  std::ostream & Write(std::ostream & os) const
  { return Dwm::StreamIO::WriteV(os, id, name, category); }
#endif
};
```

Listing 3.5: `MyType` with explicit `Dwm::StreamIO` support when needed

3.4.3 A test program to write and read `std::vector<MyType>`

The program in listing 3.6 utilizes a user-defined type MyType that has explicit Dwm::StreamIO functionality as needed. MyType meets the requirements of the HasStreamRead<T> and HasStreamWrite<T> concepts by providing the Read(std::istream &) member at lines 10-11 and the Write(std::ostream &) const member at lines 13-14 when we do not have C++26 reflection.

```
1  #include <sstream>
2  #include "DwmStreamIO.hh"
3
4  struct MyType {
5    uint32_t     id;
6    std::string  name;
7    std::string  category;
8
9  #if ! defined(DWM_CAN_USE_REFLECTION)
10   std::istream & Read(std::istream & is)
11   { return Dwm::StreamIO::ReadV(is, id, name, category); }
12
13   std::ostream & Write(std::ostream & os) const
14   { return Dwm::StreamIO::WriteV(os, id, name, category); }
15 #endif
16
17   bool operator <=> (const MyType &) const = default;  // default comparison
18 };
19
20 int main(int argc, char *argv[])
21 {
22   bool                 rc{false};
23   std::vector<MyType>  myvec1 {
24     {1, "cat", "feline"},
25     {2, "dog", "canine" },
26     {3, "cow", "bovine" }
27   };
28   std::stringstream    ss;
29   if (Dwm::StreamIO::Write(ss, myvec1)) {
30     std::vector<MyType>  myvec2;
31     if (Dwm::StreamIO::Read(ss, myvec2)) {
32       rc = (myvec2 == myvec1);
33     }
34   }
35   return rc ? 0 : 1;
36 }
```

Listing 3.6: Integrating a user-defined type

The main function creates a vector of MyType named myvec1 on line 23. It writes myvec1 to the stringstream named ss at line 29. It then creates a second vector of MyType named myvec2 on line 30 and reads the contents of myvec2 from ss on line 31. On line 32 it checks if the vector we read (myvec2) matches the vector we wrote (myvec1).

3.4.4 Skipping data when using C++26 reflection

Chapter 4

Primary I/O Classes

There are several primary I/O classes in libDwm. Each is used with a particular type of source and sink.

- `Dwm::StreamIO` is used with C++ iostreams (`std::istream` and `std::ostream`).
- `Dwm::FileIO` is used with `FILE`s crated via `fopen()`, `fdopen()`, et. al.
- `Dwm::DescriptorIO` is used with UNIX file descriptors created via `open()`, et. al.
- `Dwm::BZ2IO` is used with bz2 streams (`BZFILE`s opened via `BZ2_bzopen()`).
- `Dwm::GZIO` is used with gz streams (`gzFile`s opened via `gzopen()`).

When adding I/O support for your own data type (struct or class), `Dwm::StreamIO` support would typically be the first target. That's largely because for fixed length types, all of the other I/O classes can be supported via an intermediate pass through `Dwm::StreamIO` with a `std::stringstream` as a data buffer.

4.1 Dwm::StreamIO

source type: std::istream
sink type: std::ostream

Dwm::StreamIO provides static functions and function templates to read and write from/to C++ iostreams. Conceptually, the primary interface looks like four static member function templates:

```
namespace Dwm {
  class StreamIO {
  public:
    template <typename T>
    static std::istream & Read(std::istream & is, T & t);

    template <typename T>
    static std::ostream & Write(std::ostream & os, const T & t);

    template <typename... Args>
    static std::istream & ReadV(std::istream & is, Args & ...args);

    template <typename... Args>
    static std::ostream & WriteV(std::ostream & os, const Args & ...args);
  };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of Dwm::HasStreamRead and Dwm::HasStreamWrite.

Note that ReadV() and WriteV() are just convenient variadic function templates that will read or write multiple objects in sequence.

4.1.1 Using Dwm::StreamIO with user defined types

If a user-defined type provides the following two public members, Dwm::StreamIO will utilize them to read and write instances of the user defined type:

```
std::istream & Read(std::istream & is)
std::ostream & Write(std::ostream & os) const
```

See the Dwm::HasStreamRead and Dwm::HasStreamWrite concepts for the semantic requirements of these members.

4.2 Dwm::FileIO

source type: FILE *
sink type: FILE *

Dwm::FileIO provides static functions and function templates to read and write from/to FILEs. Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
  class FileIO {
  public:
    template <typename T>
    static size_t Read(FILE *f, T & t);

    template <typename T>
    static size_t Write(FILE *f, const T & t);

    template <typename... Args>
    static size_t ReadV(FILE *f, Args & ...args);

    template <typename... Args>
    static size_t WriteV(FILE *f, const Args & ...args);
  };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of Dwm::HasFileRead and Dwm::HasFileWrite.

Note that ReadV() and WriteV() are just convenient variadic function templates that will read or write multiple objects in sequence.

4.2.1 Using Dwm::FileIO with user defined types

If a user-defined type provides the following two public members, Dwm::FileIO will utilize them to read and write instances of the user defined type:

```
size_t Read(FILE *f)
size_t Write(FILE *f) const
```

See the Dwm::HasFileRead and Dwm::HasFileWrite concepts for the semantic requirements of these members.

4.3 Dwm::DescriptorIO

source type: int
sink type: int

Dwm::DescriptorIO provides static functions and function templates to read and write from/to UNIX file descriptors. Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
  class DescriptorIO {
  public:
    <template typename T> static ssize_t Read(int fd, T & t);

    <template typename T> static ssize_t Write(int fd, const T & t);

    template <typename ...Args>
    static ssize_t ReadV(int fd, Args & ...args);

    template <typename ...Args>
    static ssize_t WriteV(int fd, const Args & ...args);
  };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of Dwm::HasDescriptorRead and Dwm::HasDescriptorWrite.

Note that ReadV() and WriteV() are just convenient variadic function templates that will read or write multiple objects in sequence.

4.3.1 Using Dwm::DescriptorIO with user defined types

If a user defined type provides the following two public members, Dwm::DescriptorIO will utilize them to read and write instances of the user defined type:

```
ssize_t Read(int fd)
ssize_t Write(int fd) const
```

See the Dwm::HasDescriptorRead and Dwm::HasDescriptorWrite concepts for the semantic requirements of these members.

4.4 Dwm::BZ2IO

source type: BZFILE *
sink type: BZFILE *

Dwm::BZ2IO provides static functions and function templates to read and write from/to bz2 streams (BZFILE *). Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
  class BZ2IO {
  public:
    <template typename T>
    static int BZRead(BZFILE *bzf, T & t);

    <template typename T>
    static int BZWrite(BZFILE *bzf, const T & t);

    template <typename ...Args>
    static int BZReadV(BZFILE *bzf, Args & ...args);

    template <typename ...Args>
    static int BZWriteV(BZFILE *bzf, const Args & ...args);
  };
}
```

The actual implementation uses many function and function template overloads for BZRead() and BZWrite(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of Dwm::HasBZRead and Dwm::HasBZWrite.

4.4.1 Using Dwm::BZ2IO with user defined types

If a user defined type provides the following two public members, Dwm::BZ2IO will utilize them to read and write instances of the user defined type:

```
int BZRead(BZFILE *)
int BZWrite(BZFILE *) const
```

See the Dwm::HasBZRead and Dwm::HasBZWrite concepts for the semantic requirements of these member functions.

4.5 Dwm::GZIO

gzFile	source type:	gzFile
	sink type:	gzFile

Dwm::GZIO provides static functions and function templates to read and write from/to gz streams (gzFile). Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
  class GZIO {
    public:
    <template typename T>
    static int Read(gzFile gzf, T & t);

    <template typename T>
    static int Write(gzFile gzf, const T & t);

    template <typename ...Args>
    static int ReadV(gzFile gzf, Args & ...args);

    template <typename ...Args>
    static int WriteV(gzFile gzf, const Args & ...args);
  };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of Dwm::HasGZRead and Dwm::HasGZWrite.

4.5.1 Using Dwm::GZIO with user defined types

If a user defined type provides the following two public members, Dwm::GZIO will utilize them to read and write instances of the user defined type:

```
int Read(gzFile)
int Write(gzFile) const
```

See the Dwm::HasGZRead and Dwm::HasGZWrite concepts for the expected semantics of these members.

4.6 Dwm::ASIO

Boost asio sockets

Chapter 5

Directly supported data types

Directly supported data types are those types for which we have static member functions to read the type from a source and write the type to a sink. These types include integral types, floating point types, enumerated types, `std::string` and `std::vector<bool>`.

5.1 integral types

The category of types for which `std::is_integral_v<T>` evaluates to `true`.

Size	Supported type	typically aliased from
1 byte	`bool`	
	`char`	
	`int8_t`	`char`
	`uint8_t`	`unsigned char`
2 bytes	`int16_t`	`short`
	`uint16_t`	`unsigned short`
4 bytes	`int32_t`	`int` or `long`
	`uint32_t`	`unsigned int` or `unsigned long`
8 bytes	`int64_t`	`long long`
	`uint64_t`	`unsigned long long`

For multi-byte types in this category, we use an MSB-first representation on-the-wire.

5.2 floating point types

The category of types for which `std::is_floating_point_v<T>` evaluates to `true`.

Size	Supported type
4 bytes	`float`
8 bytes	`double`

We use XDR for the on-the-wire representation of `float` and `double`.

5.3 enumerated types

The category of types for which `std::is_enum_v<T>` evaluates to `true`. Note that in general, we expect an enumerated type to have an explicit underlying type so we are certain of its size.

5.4 `std::string`

`std::string` is explicitly supported.

5.5 `std::vector<bool>`

`std::vector<bool>` is explicitly supported, mainly because it's not a conforming container (which all of us in the C++ community wish was not true). In general it's unwise to use `std::vector<bool>` due to its lack of complete conformance with what's expected of an instance of `std::vector<T>`.

Chapter 6

Supported **std** type templates

6.1 std::array<class T, std::size_t N>

We can stream any instances of `std::array<T,N>` where T satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

6.2 std::atomic<class T>

We can stream any instances of `std::atomic<T>` where T satisfies the read and write concepts of the desired primary I/O class. Note this excludes cases where T is a raw pointer, `std::shared_ptr` or `std::weak_ptr`.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

6.3 std::deque<class T>

We can stream any instances of `std::deque<T>` where T satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

6.4 std::list<class T>

We can stream any instances of `std::list<T>` where T satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

6.5 std::map<class KeyType, class MappedType>

We can stream any instances of std::map<KeyType,MappedTypeT> where KeyType and MappedType satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<KeyType> Dwm::IsStreamReadable<MappedType»	Dwm::IsStreamWritable<KeyType> Dwm::IsStreamWritable<MappedType>
Dwm::FileIO	Dwm::IsFileReadable<KeyType> Dwm::IsFileReadable<MappedType>	Dwm::IsFileWritable<KeyType> Dwm::IsFileWritable<MappedType>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<KeyType> Dwm::IsDescriptorReadable<MappedType>	Dwm::IsDescriptorWritable<KeyType> Dwm::IsDescriptorWritable<MappedType>
Dwm::BZ2IO	Dwm::IsBZ2Readable<KeyType> Dwm::IsBZ2Readable<MappedType>	Dwm::IsBZ2Writable<KeyType> Dwm::IsBZ2Writable<MappedType>
Dwm::GZIO	Dwm::IsGZReadable<KeyType> Dwm::IsGZReadable<MappedType>	Dwm::IsGZWritable<KeyType> Dwm::IsGZWritable<MappedType>

6.6 std::multimap<class KeyType, class MappedType>

We can stream any instances of std::multimap<KeyType,MappedTypeT> where KeyType and MappedType satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<KeyType> Dwm::IsStreamReadable<MappedType»	Dwm::IsStreamWritable<KeyType> Dwm::IsStreamWritable<MappedType>
Dwm::FileIO	Dwm::IsFileReadable<KeyType> Dwm::IsFileReadable<MappedType>	Dwm::IsFileWritable<KeyType> Dwm::IsFileWritable<MappedType>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<KeyType> Dwm::IsDescriptorReadable<MappedType>	Dwm::IsDescriptorWritable<KeyType> Dwm::IsDescriptorWritable<MappedType>
Dwm::BZ2IO	Dwm::IsBZ2Readable<KeyType> Dwm::IsBZ2Readable<MappedType>	Dwm::IsBZ2Writable<KeyType> Dwm::IsBZ2Writable<MappedType>
Dwm::GZIO	Dwm::IsGZReadable<KeyType> Dwm::IsGZReadable<MappedType>	Dwm::IsGZWritable<KeyType> Dwm::IsGZWritable<MappedType>

6.7 std::optional<class T>

We can stream any instances of std::optional<T> where T satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

6.8 std::pair<class FirstType, class SecondType>

We can stream any instances of std::pair<FirstType,SecondType> where FirstType and SecondType satisfy the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<FirstType> Dwm::IsStreamReadable<SecondType»	Dwm::IsStreamWritable<FirstType> Dwm::IsStreamWritable<SecondType>
Dwm::FileIO	Dwm::IsFileReadable<FirstType> Dwm::IsFileReadable<SecondType>	Dwm::IsFileWritable<FirstType> Dwm::IsFileWritable<SecondType>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<FirstType> Dwm::IsDescriptorReadable<SecondType>	Dwm::IsDescriptorWritable<FirstType> Dwm::IsDescriptorWritable<SecondType>
Dwm::BZ2IO	Dwm::IsBZ2Readable<FirstType> Dwm::IsBZ2Readable<SecondType>	Dwm::IsBZ2Writable<FirstType> Dwm::IsBZ2Writable<SecondType>
Dwm::GZIO	Dwm::IsGZReadable<FirstType> Dwm::IsGZReadable<SecondType>	Dwm::IsGZWritable<FirstType> Dwm::IsGZWritable<SecondType>

6.9 std::set<class T>

We can stream any instances of std::set<T> where T satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

6.10 std::multiset<class T>

We can stream any instances of std::multiset<T> where T satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

6.11 std::unique_ptr<class T>

We can stream instances of std::unique_ptr<T> where T is streamable and the unique_ptr's deleter_type is std::default_delete<T>. We use the deleter_type test to infer that the std::unique_ptr<T> points to a single object and not to an array. This can of course be incorrect, and hence should not be relied on as a general mechanism.

6.12 std::unordered_map<class KeyType, class MappedType>

We can stream any instances of std::unordered_map<KeyType,MappedTypeT> where KeyType and MappedType satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<KeyType> Dwm::IsStreamReadable<MappedType»	Dwm::IsStreamWritable<KeyType> Dwm::IsStreamWritable<MappedType>
Dwm::FileIO	Dwm::IsFileReadable<KeyType> Dwm::IsFileReadable<MappedType>	Dwm::IsFileWritable<KeyType> Dwm::IsFileWritable<MappedType>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<KeyType> Dwm::IsDescriptorReadable<MappedType>	Dwm::IsDescriptorWritable<KeyType> Dwm::IsDescriptorWritable<MappedType>
Dwm::BZ2IO	Dwm::IsBZ2Readable<KeyType> Dwm::IsBZ2Readable<MappedType>	Dwm::IsBZ2Writable<KeyType> Dwm::IsBZ2Writable<MappedType>
Dwm::GZIO	Dwm::IsGZReadable<KeyType> Dwm::IsGZReadable<MappedType>	Dwm::IsGZWritable<KeyType> Dwm::IsGZWritable<MappedType>

6.13 std::unordered_multimap<class KeyType, class MappedType>

We can stream any instances of std::unordered_multimap<KeyType,MappedTypeT> where KeyType and MappedType satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<KeyType> Dwm::IsStreamReadable<MappedType»	Dwm::IsStreamWritable<KeyType> Dwm::IsStreamWritable<MappedType>
Dwm::FileIO	Dwm::IsFileReadable<KeyType> Dwm::IsFileReadable<MappedType>	Dwm::IsFileWritable<KeyType> Dwm::IsFileWritable<MappedType>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<KeyType> Dwm::IsDescriptorReadable<MappedType>	Dwm::IsDescriptorWritable<KeyType> Dwm::IsDescriptorWritable<MappedType>
Dwm::BZ2IO	Dwm::IsBZ2Readable<KeyType> Dwm::IsBZ2Readable<MappedType>	Dwm::IsBZ2Writable<KeyType> Dwm::IsBZ2Writable<MappedType>
Dwm::GZIO	Dwm::IsGZReadable<KeyType> Dwm::IsGZReadable<MappedType>	Dwm::IsGZWritable<KeyType> Dwm::IsGZWritable<MappedType>

6.14 std::unordered_set<class T>

We can stream any instances of std::unordered_set<T> where T satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

6.15 std::unordered_multiset<class T>

We can stream any instances of std::unordered_multiset<T> where T satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

6.16 std::vector<class T>

We can stream any instances of std::vector<T> where T satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
Dwm::StreamIO	Dwm::IsStreamReadable<T>	Dwm::IsStreamWritable<T>
Dwm::FileIO	Dwm::IsFileReadable<T>	Dwm::IsFileWritable<T>
Dwm::DescriptorIO	Dwm::IsDescriptorReadable<T>	Dwm::IsDescriptorWritable<T>
Dwm::BZ2IO	Dwm::IsBZ2Readable<T>	Dwm::IsBZ2Writable<T>
Dwm::GZIO	Dwm::IsGZReadable<T>	Dwm::IsGZWritable<T>

Chapter 7

Concepts

7.1 Streamable

7.2 Concepts used by `Dwm::StreamIO` (C++ iostreams)

7.2.1 `HasStreamRead<T>`

Evaluates to true if T has a member function with signature:

```
std::istream & Read(std::istream & is)
```

Required semantics

- Must populate `*this` with contents read from `is`
- Must return `is`

Other semantics

- May set `std::ios_base::failbit`

7.2.2 `HasStreamWrite<T>`

Evaluates to true if T has a member function with signature:

```
std::ostream & Write(std::ostream & os) const
```

Required semantics

- Must write the contents of `*this` to `os`
- Must return `os`

Other semantics

- May set `std::ios_base::failbit`

7.2.3 `Dwm::IsStreamReadable<T>`

Evaluates to true if T is a directly supported type, a type that satisfies `HasStreamRead<T>`, an instance of a supported class template, or (if C++26 reflection is available) is readable via reflection.

7.2.4 `Dwm::IsStreamWritable<T>`

Evaluates to true if T is a directly supported type, a type that satisfies `HasStreamWrite<T>`, an instance of a supported class template, or (if C++26 reflection is available) is writable via reflection.

7.3 Concepts used by `Dwm::FileIO` (`FILE`s, i.e. `cstdio`)

7.3.1 `HasFileRead<T>`

Evaluates to true if T has a member function with signature:

```
size_t Read(FILE *f)
```

Required semantics

- Must populate `*this` with contents read from `f`
- Must return a non-zero `size_t` on success, `0` on failure

7.3.2 `HasFileWrite<T>`

Evaluates to true if T has a member function with signature:

```
size_t Write(FILE *f) const
```

Required semantics

- Must write the contents of `*this` to `f`
- Must return a non-zero `size_t` on success, `0` on failure

7.3.3 `Dwm::IsFileReadable<T>`

7.3.4 `Dwm::IsFileWritable<T>`

7.4 Concepts used by `Dwm::DescriptorIO` (UNIX descriptors)

7.4.1 `HasDescriptorRead<T>`

Evaluates to true if T has a member function with signature:

```
ssize_t Read(int fd)
```

Required semantics

- Must populate `*this` with contents read from `fd`
- Must return a positive `ssize_t` on success, `-1` on failure

7.4.2 `HasDescriptorWrite<T>`

Evaluates to true if T has a member function with signature:

```
ssize_t Write(int fd) const
```

Required semantics

- Must write the contents of `*this` to `fd`
- Must return a positive `ssize_t` on success, `-1` on failure

7.4.3 `Dwm::IsDescriptorReadable<T>`

7.4.4 `Dwm::IsDescriptorWritable<T>`

7.5 Boost asio sockets

7.5.1 HasAsioRead<T>

7.5.2 HasAsioWrite<T>

7.6 Concepts used by Dwm::BZ2IO (BZFILEs from BZ2_bzopen())

7.6.1 HasBZRead<T>

Evaluates to true if T has a member function with signature:

```
int BZRead(BZFILE *bzf)
```

Required semantics

- Must populate `*this` with contents read from `bzf`
- Must return a positive `int` on success, `-1` on failure

7.6.2 HasBZWrite<T>

Evaluates to true if T has a member function with signature:

```
int Write(BZFILE *bzf) const
```

Required semantics

- Must write the contents of `*this` to `bzf`
- Must return a positive `int` on success, `-1` on failure

7.6.3 Dwm::IsBZ2Readable<T>

7.6.4 Dwm::IsBZ2Writable<T>

7.7 Concepts used by Dwm::GZIO (gzFiles from gzopen())

7.7.1 HasGZRead<T>

Evaluates to true if T has a member function with signature:

```
int Read(gzFile gzf)
```

Required semantics

- Must populate `*this` with contents read from `gzf`
- Should return the number of decompressed bytes read on success, -1 on failure

7.7.2 HasGZWrite<T>

Evaluates to true if T has a member function with signature:

```
int Write(gzFile gzf) const
```

Required semantics

- Must write the contents of `*this` to `gzf`
- Should return the number of uncompressed bytes written on success, -1 on failure

7.7.3 Dwm::IsGZReadable<T>

7.7.4 Dwm::IsGZWritable<T>

7.8 HasSkipAnnotation<std::meta::info info>

Evaluates to true if the given reflection `info` has an annotation of the form `[[=Dwm::skip_io]]`

This requires C++26 reflection, specifically P2996 and P3394.

7.9 Streamed length

7.9.1 Constant streamed length

7.9.2 Constant streamed length via `static constexpr uint64_t StreamedLength()` member

7.9.3 Constant streamed length from C++26 reflection

When using C++26 reflection for serialization and deserialization of instances of a given type, we can determine at compile time whether or not the streamed length of the type is constant, and hence whether or not we can leverage `StreamIO` to provide `DescriptorIO`, `FileIO`, `BZ2IO`, `GZIO` and `ASIO` functionality automatically (via a pass through a `std::stringstream`).

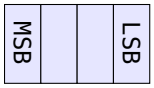
Chapter 8

On-the-wire formats

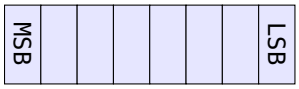
8.1 On-the-wire formats, network byte order (MSB first)

8.1.1 16 bit integral types (**int16_t**, **uint16_t**)

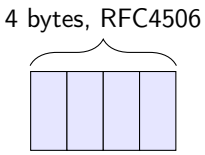
8.1.2 32 bit integral types (**int32_t**, **uint32_t**)



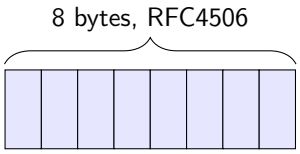
8.1.3 64 bit integral types (**int64_t**, **uint64_t**)



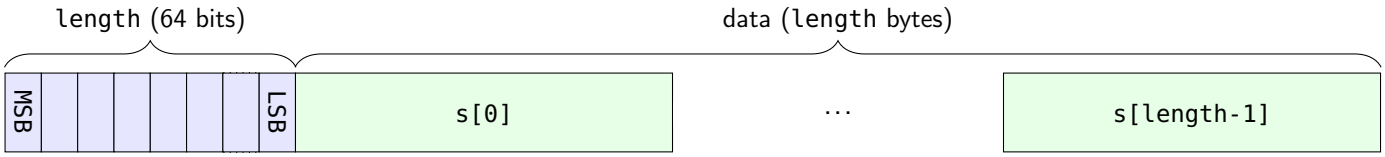
8.1.4 **float**



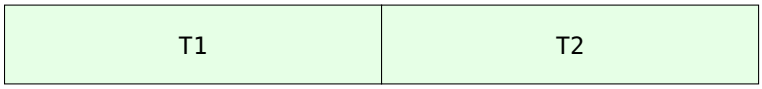
8.1.5 **double**



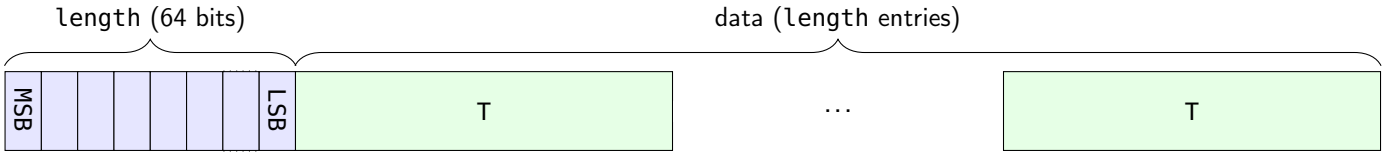
8.1.6 std::string



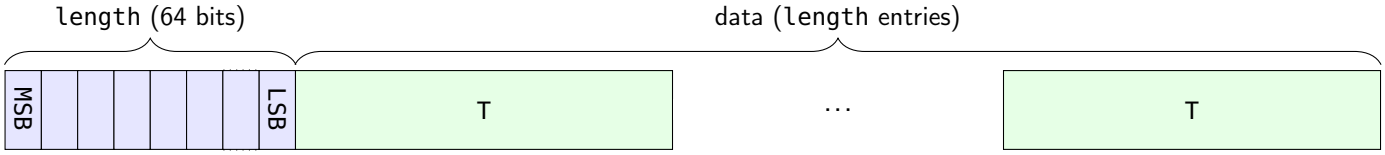
8.1.7 std::pair<T1,T2>



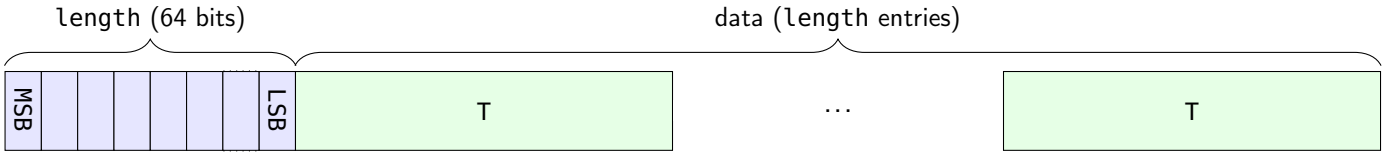
8.1.8 std::vector<T>



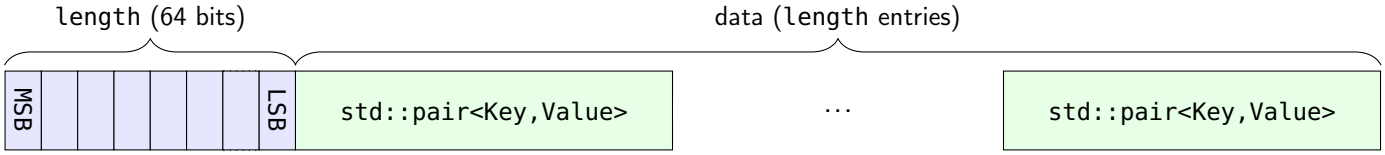
8.1.9 std::deque<T>



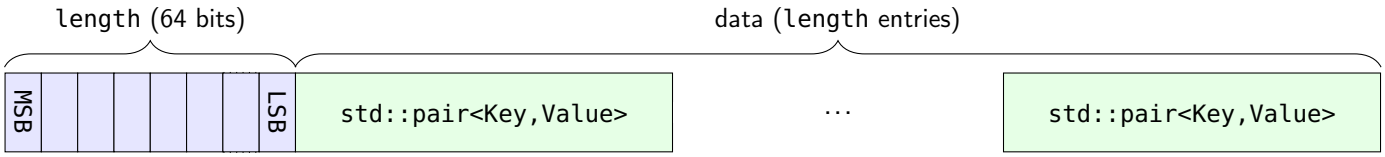
8.1.10 std::list<T>



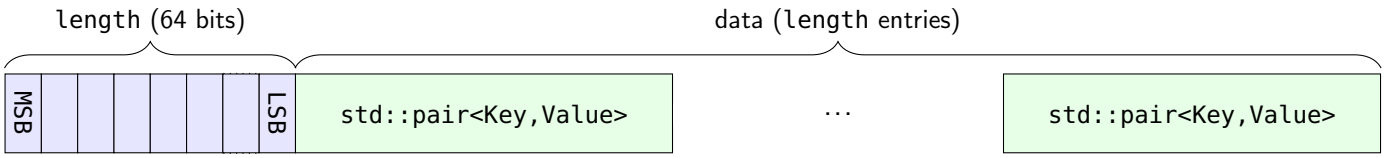
8.1.11 std::map<Key,Value>



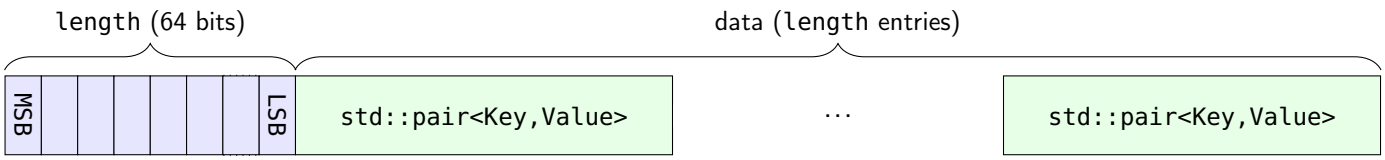
8.1.12 std::multimap<Key,Value>



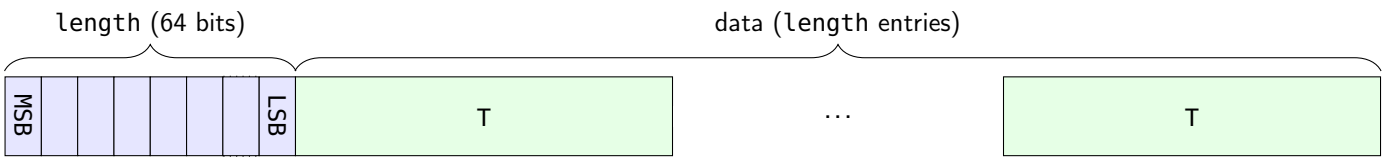
8.1.13 std::unordered_map<Key,Value>



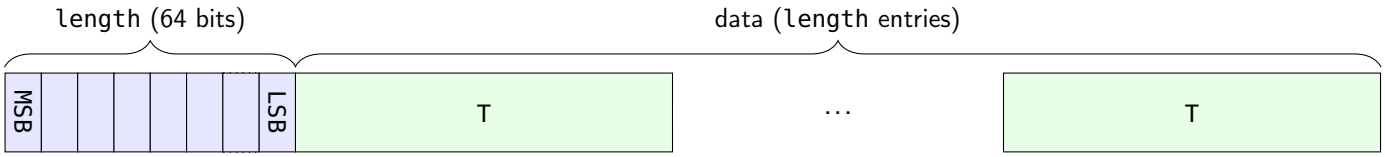
8.1.14 std::unordered_multimap<Key,Value>



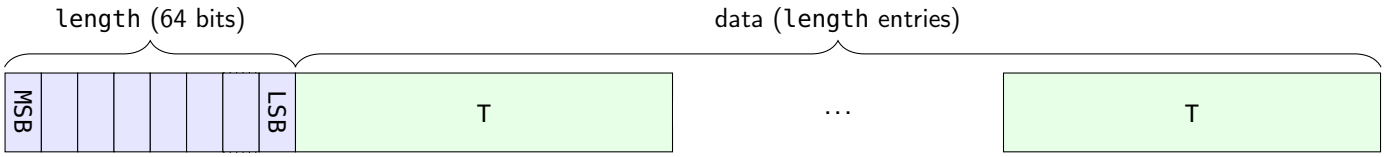
8.1.15 std::set<T>



8.1.16 `std::multiset<T>`



8.1.17 `std::unordered_set<T>`



8.1.18 `std::unordered_multiset<T>`